

CANARA ENGINEERING COLLEGE

Benjanapadavu, Mangaluru, Karnataka

Department of Computer Science and Business Systems

MINI PROJECT REPORT

on

HIGH-SPEED ZERO-COPY DATA TRANSFER SYSTEM

A Linux-based high-performance data transfer architecture using XDP,
eBPF, AF_XDP, shared buffers and asynchronous I/O

Submitted by

1. STUDENT NAME 1
USN: XXXXXXXXX
2. STUDENT NAME 2
USN: XXXXXXXXX
3. STUDENT NAME 3
USN: XXXXXXXXX

Under the Guidance of

GUIDE NAME

Designation

Academic Year 2026–27

CERTIFICATE

This is to certify that the mini project entitled “**High-Speed Zero-Copy Data Transfer System**” is a bonafide work carried out by the students listed below in partial fulfillment of the requirements for the mini project during the academic year 2026–27.

Sl. No.	Student Name	USN
1	STUDENT NAME 1	XXXXXXXXX
2	STUDENT NAME 2	XXXXXXXXX
3	STUDENT NAME 3	XXXXXXXXX

Project Guide

Signature: _____

Head of Department

Signature: _____

DECLARATION

We hereby declare that the mini project entitled “**High-Speed Zero-Copy Data Transfer System**” is our original work carried out as part of the academic requirements during the academic year 2026–27.

The work presented in this report has been developed by the project team and has not been submitted previously for the award of any other degree, diploma, or academic qualification.

STUDENT NAME 1

STUDENT NAME 2

STUDENT NAME 3

ACKNOWLEDGEMENT

We express our sincere gratitude to the Department of Computer Science and Business Systems, Canara Engineering College, for providing us with the opportunity to undertake this mini project.

We would like to thank our project guide and faculty members for their valuable guidance, technical support and encouragement throughout the development of this project.

We also acknowledge the Linux kernel and open-source development communities whose documentation and technologies form the foundation of this project.

Finally, we thank our friends and classmates for their support and valuable feedback during the development and testing of the system.

ABSTRACT

Modern applications increasingly handle large amounts of data in the form of video streams, files, scientific datasets and network traffic. Conventional data-transfer paths may involve repeated copies of the same payload between kernel buffers, application buffers, network buffers and storage buffers. These operations can increase CPU utilization, memory bandwidth consumption and latency.

This project investigates a Linux-based high-speed data-transfer architecture designed to reduce unnecessary data movement. The proposed system combines technologies such as XDP, eBPF, AF_XDP, shared memory, reusable buffers, ring buffers and asynchronous I/O using `io_uring`.

A central idea of the system is to treat buffers as reusable resources and transfer buffer descriptors or ownership information between processing stages whenever possible instead of repeatedly copying the underlying payload.

The system supports two data-sharing modes: Host-to-Host and Host-to-Many. In Host-to-Host communication, data from one host is transferred to another host through the server data path. In Host-to-Many communication, a publisher can distribute data to multiple receiving hosts.

The system will be evaluated using throughput, latency, CPU utilization, memory usage, copy count and scalability.

Contents

Certificate	1
Declaration	2
Acknowledgement	3
Abstract	4
1 Introduction	10
1.1 Background	10
1.2 Motivation	10
1.3 Problem Statement	11
1.4 Objectives	11
2 Literature Survey	12
2.1 Introduction	12
2.2 io_uring	12
2.3 XDP and eBPF	12
2.4 AF_XDP	13
2.5 ChainIO	13
2.6 Z-Stack	13
2.7 Comparison	14
3 Research Gap	15
3.1 Research Gap	15

4 Existing System	17
4.1 Conventional Data Path	17
4.2 Limitations	17
5 Proposed System	18
5.1 Overview	18
5.2 High-Level Architecture	19
5.3 Core Concept	19
5.4 Host-to-Host	20
5.5 Host-to-Many	20
6 System Modules	21
6.1 Network and Data Ingestion	21
6.2 Buffer Manager	21
6.3 Storage and I/O	21
6.4 Server	22
6.5 Client	22
6.6 Performance Evaluation	22
7 Methodology	23
7.1 Data Flow	23
7.2 Buffer Lifecycle	23
8 Technologies Used	25
9 Implementation	26
9.1 Buffer Management	26
9.2 Data Chunking	26
9.3 Data Reconstruction	26
10 Experimental Setup	27
10.1 Hardware	27

10.2 Software	27
11 Performance Evaluation	28
11.1 Metrics	28
11.2 Throughput	28
11.3 Latency	28
11.4 Comparison	29
11.5 Results	29
12 Comparison with Related Systems	30
12.1 ChainIO	30
12.2 Z-Stack	30
12.3 Comparison Table	31
13 Advantages and Limitations	32
13.1 Advantages	32
13.2 Limitations	32
14 Future Scope	33
15 Conclusion	34
16 Team Contribution	35

List of Figures

5.1	High-level architecture of the proposed system	19
-----	--	----

List of Tables

2.1	Comparison of Related Systems	14
8.1	Technology Stack	25
10.1	Hardware Configuration	27
10.2	Software Configuration	27
11.1	Performance Results	29
12.1	Comparison with Related Systems	31
16.1	Team Contribution	35

Chapter 1

Introduction

1.1 Background

High-speed data movement is an important requirement in modern computing systems. Applications such as video streaming, distributed storage, cloud computing and scientific computing continuously transfer large volumes of data.

In a conventional data-transfer architecture, data may pass through multiple intermediate buffers. For example, data received from a network interface may be transferred to kernel buffers, application buffers and processing buffers before reaching the destination.

When the same payload is copied multiple times, CPU utilization and memory bandwidth consumption increase.

This project investigates an alternative approach in which reusable buffers and descriptors are used to reduce unnecessary payload movement.

1.2 Motivation

The main motivation of this project is the increasing size of modern data workloads.

Large video files, datasets and network streams may contain hundreds of megabytes or several gigabytes of data. Repeatedly copying such data can introduce significant overhead.

The project therefore investigates:

1. Reusable data buffers.
2. Shared memory.

3. Ring buffers.
4. Descriptor-based data movement.
5. Buffer ownership transfer.
6. High-speed network processing.
7. Asynchronous I/O.

1.3 Problem Statement

Traditional data-transfer architectures may perform multiple payload copies between network, memory, processing and storage subsystems.

The problem addressed by this project is:

How can a Linux-based data-transfer system reduce unnecessary payload copying across multiple processing stages while maintaining correct buffer ownership and efficient communication between hosts?

1.4 Objectives

The objectives of the project are:

1. To develop a high-speed Linux-based data-transfer system.
2. To reduce unnecessary payload copies.
3. To investigate XDP, eBPF and AF_XDP.
4. To implement reusable buffers.
5. To investigate shared memory and memory mapping.
6. To use ring buffers for efficient data exchange.
7. To investigate asynchronous I/O using `io_uring`.
8. To implement buffer ownership management.
9. To support Host-to-Host data transfer.
10. To support Host-to-Many data distribution.
11. To evaluate system performance.

Chapter 2

Literature Survey

2.1 Introduction

High-performance data movement has been studied extensively in networking and operating-system research. Several approaches aim to reduce data copies, kernel-user transitions and protocol processing overhead.

The technologies and systems most relevant to this project are `io_uring`, XDP, AF_XDP, ChainIO and Z-Stack.

2.2 `io_uring`

`io_uring` is a Linux asynchronous I/O interface that provides submission and completion queues shared between applications and the kernel.

It allows applications to submit multiple I/O operations and process their completions asynchronously.

This can reduce the overhead associated with repeatedly issuing individual blocking system calls.

However, `io_uring` alone does not provide a complete zero-copy data path between storage and networking.

2.3 XDP and eBPF

eXpress Data Path (XDP) provides an early packet-processing point in the Linux networking path.

XDP programs can inspect, modify, redirect or drop packets before they traverse a large portion of the traditional networking stack.

eBPF provides the programmable execution framework used by XDP and other Linux subsystems.

These technologies are relevant to this project because they provide mechanisms for efficient packet processing.

2.4 AF_XDP

AF_XDP provides a high-performance interface for transferring network packets between the Linux networking subsystem and user-space applications.

It is designed for applications requiring efficient packet processing and high packet rates.

AF_XDP primarily addresses the networking portion of the data path. Additional mechanisms are required to connect network data efficiently with storage and application-level processing.

2.5 ChainIO

ChainIO is an eBPF-based system designed to improve data movement between storage and networking.

It connects `io_uring` and AF_XDP through descriptors, ring buffers and kernel-assisted orchestration.

The system addresses the problem of moving data from disk to network while avoiding unnecessary intermediate operations.

ChainIO is particularly relevant to this project because it demonstrates how storage and networking paths can be connected using descriptor-based mechanisms.

2.6 Z-Stack

Z-Stack is a high-performance userspace TCP/IP protocol stack based on DPDK.

It implements a zero-copy socket architecture and uses shared memory mechanisms to reduce unnecessary data movement during network processing.

Z-Stack demonstrates the performance potential of userspace networking and zero-copy communication.

Unlike Z-Stack, the proposed project does not aim to implement a complete userspace TCP/IP stack. Instead, it focuses on reusable buffers, ownership management and data distribution.

2.7 Comparison

Table 2.1: Comparison of Related Systems

System	Main Focus	Technology	Relevance
io_uring	Asynchronous I/O	Linux kernel	Efficient asynchronous storage I/O
XDP	Packet processing	eBPF / XDP	Early network processing
AF_XDP	Packet transfer	Linux / eBPF	High-speed packet I/O
ChainIO	Storage to network	eBPF, io_uring, AF_XDP	Descriptor-based data path
Z-Stack	High-speed networking	DPDK, userspace TCP/IP	Zero-copy networking
Proposed System	Cross-stage data movement	XDP, eBPF, AF_XDP, shared buffers, io_uring	Reusable buffers, ownership and Host-to-Many distribution

Chapter 3

Research Gap

3.1 Research Gap

The reviewed technologies address different parts of the high-performance data-transfer problem.

`io_uring` provides asynchronous I/O, while XDP and AF_XDP provide efficient network processing.

Z-Stack demonstrates zero-copy userspace networking using DPDK and a userspace TCP/IP stack.

ChainIO connects storage and networking using eBPF, `io_uring`, AF_XDP and descriptor-based mechanisms.

Based on these approaches, this project investigates a buffer-oriented architecture in which buffers are explicitly managed as reusable resources.

The proposed system focuses on:

1. Explicit buffer ownership.
2. Buffer lifecycle management.
3. Reusable shared buffers.
4. Ring-buffer based descriptor exchange.
5. Host-to-Host communication.
6. Host-to-Many distribution.
7. Data reconstruction at receiving hosts.

The purpose is not to assume that every operation can be made zero-copy. Instead, the system experimentally evaluates where payload copies can be eliminated or reduced.

Chapter 4

Existing System

4.1 Conventional Data Path

A conventional data-transfer path may be represented as:

Storage → Kernel Buffer → Application Buffer → Network Buffer → Destination

Depending on the implementation, multiple copies of the same payload may occur.

4.2 Limitations

The limitations include:

1. Increased CPU utilization.
2. Increased memory bandwidth usage.
3. Additional latency.
4. Repeated buffer allocation.
5. Increased memory traffic.
6. Difficulty in efficiently distributing large data to multiple receivers.

Chapter 5

Proposed System

5.1 Overview

The proposed system is a Linux-based high-speed data-transfer architecture focused on reducing unnecessary data movement.

The system consists of:

1. Network and Data Ingestion.
2. Buffer and Data-Path Manager.
3. Storage and Asynchronous I/O.
4. Server and Distribution Layer.
5. Client and Data Reconstruction.
6. Performance Evaluation.

5.2 High-Level Architecture

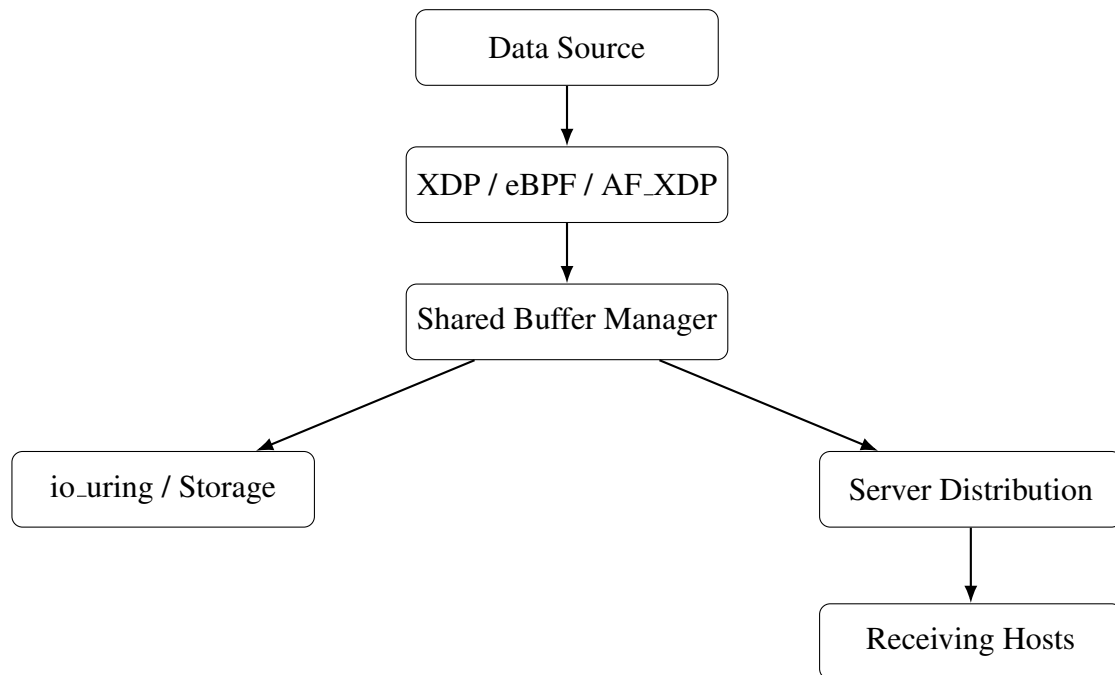


Figure 5.1: High-level architecture of the proposed system

5.3 Core Concept

Instead of repeatedly copying the payload:

Copy → Copy → Copy

the system attempts to use:

Buffer → Descriptor → Ownership Transfer → Same Buffer

A descriptor can contain:

- Buffer identifier.
- Data offset.
- Data length.
- Sequence number.
- Owner.

- Buffer state.
- Generation number.

5.4 Host-to-Host

The Host-to-Host mode is:

$$Host_A \rightarrow Server \rightarrow Host_B$$

Host A publishes data and Host B receives and reconstructs it.

5.5 Host-to-Many

The Host-to-Many mode is:

$$Host_A \rightarrow Server \rightarrow \{Host_B, Host_C, \dots, Host_N\}$$

This mode allows a single publisher to distribute data to multiple receivers.

Chapter 6

System Modules

6.1 Network and Data Ingestion

This module handles incoming network traffic using technologies such as XDP, eBPF and AF_XDP.

6.2 Buffer Manager

The buffer manager handles:

- Buffer allocation.
- Buffer reuse.
- Buffer ownership.
- Ring buffers.
- Shared memory.
- Synchronization.

6.3 Storage and I/O

The storage module handles file operations and investigates asynchronous I/O using `io_uring`.

6.4 Server

The server manages connections and distributes data between participating hosts.

6.5 Client

The client receives data chunks and reconstructs the original data.

6.6 Performance Evaluation

This module measures:

- Throughput.
- Latency.
- CPU utilization.
- Memory usage.
- Copy count.
- Scalability.

Chapter 7

Methodology

7.1 Data Flow

The proposed methodology consists of:

1. Receive data.
2. Process network traffic.
3. Place data into reusable buffers.
4. Assign buffer metadata.
5. Transfer descriptors between components.
6. Transfer ownership when required.
7. Distribute data.
8. Reconstruct data at receivers.
9. Release and reuse buffers.
10. Measure performance.

7.2 Buffer Lifecycle

The conceptual buffer lifecycle is:

FREE → FILLED → IN USE → TRANSFERRED → RELEASED → FREE

This lifecycle allows buffers to be reused instead of repeatedly allocated.

Chapter 8

Technologies Used

Table 8.1: Technology Stack

Category	Technology
Operating System	Linux
Programming	C / C++
Networking	XDP / eBPF / AF_XDP
Memory	Shared Memory / mmap
Data Structures	Ring Buffers
Storage I/O	io_uring
Networking Interface	TCP / UDP / Sockets
Performance	perf / Custom Benchmarks

Chapter 9

Implementation

9.1 Buffer Management

Reusable buffers are maintained by the buffer manager. Each buffer is associated with metadata describing its current state and ownership.

9.2 Data Chunking

Large data streams are divided into smaller chunks.

Each chunk can contain:

- Sequence number.
- Buffer identifier.
- Payload size.
- Ownership information.
- State information.

9.3 Data Reconstruction

The receiver uses the sequence numbers to reconstruct the original data.

$$C_0, C_1, C_2, \dots, C_n \rightarrow \textit{Original Data}$$

Chapter 10

Experimental Setup

10.1 Hardware

The following information will be filled after the final experimental setup is fixed.

Table 10.1: Hardware Configuration

Processor	To be filled
RAM	To be filled
Storage	To be filled
Network Interface	To be filled
Network Speed	To be filled
Number of Hosts	To be filled

10.2 Software

Table 10.2: Software Configuration

Operating System	Linux
Kernel Version	To be filled
Compiler	GCC / G++
Programming Language	C / C++
eBPF/XDP	To be filled
io_uring	Linux Kernel Interface

Chapter 11

Performance Evaluation

11.1 Metrics

The system will be evaluated using:

1. Throughput.
2. End-to-end latency.
3. CPU utilization.
4. Memory utilization.
5. Number of payload copies.
6. Buffer reuse.
7. Scalability.

11.2 Throughput

$$Throughput = \frac{Total\ Data\ Transferred}{Transfer\ Time}$$

11.3 Latency

$$Latency = T_{destination} - T_{source}$$

11.4 Comparison

The following configurations will be evaluated:

1. Conventional copy-based transfer.
2. Low-copy transfer.
3. Proposed reusable-buffer architecture.

11.5 Results

Actual measured results will be added after completion of the prototype.

Table 11.1: Performance Results

Metric	Baseline	Low-Copy	Proposed
Throughput	—	—	—
Latency	—	—	—
CPU Usage	—	—	—
Memory Usage	—	—	—
Copy Count	—	—	—

Chapter 12

Comparison with Related Systems

12.1 ChainIO

ChainIO focuses on connecting storage and network data paths using eBPF, `io_uring`, AF_XDP and descriptor-based mechanisms.

The proposed project is related because it also investigates efficient movement of data between different processing stages.

The project additionally focuses on explicit reusable-buffer ownership and Host-to-Host and Host-to-Many data distribution.

12.2 Z-Stack

Z-Stack focuses on high-performance userspace networking using DPDK and a userspace TCP/IP stack.

It demonstrates how zero-copy networking can be achieved within a high-performance userspace networking architecture.

The proposed project differs by focusing on a Linux-based data-transfer architecture rather than implementing a complete userspace TCP/IP stack.

12.3 Comparison Table

Table 12.1: Comparison with Related Systems

Feature	ChainIO	Z-Stack	Proposed
eBPF	Yes	No	Yes
AF_XDP	Yes	No	Yes
io_uring	Yes	No	Yes
DPDK	No	Yes	No
Userspace	No	Yes	No
TCP/IP			
Shared Buffers	Yes	Yes	Yes
Descriptor Based	Yes	Yes	Yes
Buffer Ownership	–	–	Yes
Host-to-Host	Related	Yes	Yes
Host-to-Many	–	–	Yes

Chapter 13

Advantages and Limitations

13.1 Advantages

1. Reduced unnecessary payload movement.
2. Reusable memory buffers.
3. Explicit buffer lifecycle.
4. Host-to-Host support.
5. Host-to-Many support.
6. Integration of networking and storage technologies.
7. Performance can be experimentally evaluated.

13.2 Limitations

1. Not every stage can necessarily be zero-copy.
2. Buffer ownership requires synchronization.
3. The implementation is Linux dependent.
4. Performance depends on hardware and workload.
5. Network transmission between physical hosts still requires network transmission of the data.

Chapter 14

Future Scope

Future improvements may include:

1. More advanced buffer ownership protocols.
2. NUMA-aware buffer allocation.
3. Hardware network offloading.
4. Larger Host-to-Many deployments.
5. Improved failure recovery.
6. Adaptive data paths.
7. Integration with additional kernel-bypass technologies.

Chapter 15

Conclusion

This project presents a Linux-based high-speed data-transfer architecture focused on reducing unnecessary data movement.

The system combines XDP, eBPF, AF_XDP, shared buffers, ring buffers and asynchronous I/O.

A central idea is to treat buffers as reusable resources and transfer descriptors and ownership between processing stages instead of repeatedly copying the underlying payload.

The system supports Host-to-Host and Host-to-Many communication.

The final prototype will be evaluated using throughput, latency, CPU utilization, memory usage, copy count and scalability.

Chapter 16

Team Contribution

The project is developed by a team of three members.

Table 16.1: Team Contribution

Member	Area	Responsibilities
Member 1	Network	XDP, eBPF, AF_XDP and packet processing
Member 2	Buffer Management	Shared memory, mmap, ring buffers and ownership
Member 3	Server and I/O	Server, clients, io_uring and performance evaluation

All members participate in integration, testing, debugging and documentation.

Bibliography

- [1] Z. Cao, X. He, Y. Hu, Y. Zheng, Y. Yang, J. Su, W. Zhang, and A. Quinn,
ChainIO: Bridging Disk and Network Domains with eBPF,
Proceedings of the 3rd Workshop on eBPF and Kernel Extensions, 2025.
<https://doi.org/10.1145/3748355.3748371>
- [2] A. B. Narappa, F. Parola, S. Qi, and K. K. Ramakrishnan,
Z-Stack: A High-Performance DPDK-Based Zero-Copy TCP/IP Protocol Stack,
2024 IEEE 30th International Symposium on Local and Metropolitan Area Networks.
<https://doi.org/10.1109/LANMAN61958.2024.10621881>
- [3] Linux Kernel Documentation,
io_uring.
https://docs.kernel.org/io_uring/
- [4] Linux Kernel Documentation,
AF_XDP.
https://docs.kernel.org/networking/af_xdp.html
- [5] eBPF Documentation,
eBPF Documentation.
<https://docs.ebpf.io/>
- [6] DPDK Project,
Data Plane Development Kit.
<https://www.dpdk.org/>
- [7] Aravind066390,
Mini Project 2026–27.
https://github.com/Aravind066390/Mini_project____2026-27